VERİ BİLİMCİ MÜFREDATI

05 | SQL

Ayrıntılı Konu Anlatımı

Veri çekme • Filtreleme • JOIN • GROUP BY • CTE • Window Functions • Analitik SQL

Amaç
SQL sorgusunu ezberlemek yerine veriyi ilişkisel olarak düşünmek, doğru grain ile sorgu kurmak, iş sorularını analitik SQL ile yanıtlamak ve mülakat seviyesinde güvenli sorgu yazmak.

Data Science / Analytics Track

1. SQL neden veri bilimci için temel beceridir?

Gerçek şirket verisi çoğunlukla CSV dosyası halinde beklemez; ilişkisel veritabanlarında, veri ambarlarında veya lakehouse tablolarında tutulur. Veri bilimci model kurmadan önce doğru evreni çıkarmalı, doğru joinleri yapmalı, metrikleri hesaplamalı ve veri kalitesini doğrulamalıdır. SQL bu nedenle yalnızca veri çekme dili değil, veri düşünme dilidir.

İş ihtiyacı	SQL karşılığı	Örnek
Belirli kayıtları bulmak	WHERE	Son 30 gündeki siparişler
Özet metrik hesaplamak	GROUP BY	Şehir bazında gelir
Tabloları birleştirmek	JOIN	Müşteri + sipariş
Satırları kıyaslamak	Window Function	Müşterinin önceki siparişi
Karmaşık analizi parçalamak	CTE	Cohort/funnel analizi
Veri kalitesini ölçmek	COUNT, CASE	Null/duplicate oranı

Temel zihniyet
SQL yazarken ilk soru “hangi keyword?” değil, “çıktının bir satırı neyi temsil edecek?” olmalıdır. Buna grain denir. Yanlış grain, doğru görünen ama yanlış sonuç veren sorguların en yaygın nedenidir.

2. İlişkisel veri modeli ve grain

Bir tabloyu sorgulamadan önce primary key, foreign key ve satır düzeyini anlamalısın. orders tablosunda bir satır bir sipariş, order_items tablosunda bir satır bir sipariş kalemi olabilir. Bu iki tabloyu doğrudan birleştirirsen sipariş satırları çoğalabilir.

Tablo	Grain	Anahtar
customers	1 satır = 1 müşteri	customer_id
orders	1 satır = 1 sipariş	order_id
order_items	1 satır = 1 sipariş kalemi	order_id + product_id
products	1 satır = 1 ürün	product_id

```
SELECT COUNT(*) AS rows,
    COUNT(DISTINCT order_id) AS orders
FROM order_items;
```

Kontrol
Bir JOIN sonrasında COUNT(*) beklenmedik şekilde büyüyorsa önce cardinality ve grain kontrolü yap. “Many-to-many explosion” analitik hataların klasik kaynağıdır.

3. SELECT - veri seçmenin temeli

SELECT hangi kolonları veya hesaplanmış ifadeleri döndüreceğini belirler. Üretim sorgularında SELECT * yerine ihtiyaç duyulan kolonları açıkça seçmek daha güvenli ve okunabilirdir.

```
SELECT
  order_id,
  customer_id,
  total_amount,
  total_amount * 0.20 AS estimated_tax
FROM orders;
```

- Alias için AS kullan: total_amount AS revenue.
- DISTINCT tekrar eden kombinasyonları kaldırır; veri kalitesi problemlerini gizlemek için rastgele kullanılmamalıdır.
- Hesaplamalarda NULL davranışını bil: NULL ile yapılan çoğu aritmetik işlem NULL üretir.

4. WHERE - satır filtreleme

```
SELECT order_id, customer_id, total_amount
FROM orders
WHERE order_date >= '2026-01-01'
 AND status = 'completed'
 AND total_amount > 500;
```

Operatör	Amaç	Örnek
=, <>, >, >=	Karşılaştırma	amount >= 500
BETWEEN	Aralık	amount BETWEEN 100 AND 500
IN	Küme üyeliği	country IN ('TR','DE')
LIKE	Pattern	email LIKE '%@gmail.com'
IS NULL	Eksik değer	refund_date IS NULL
AND / OR / NOT	Mantıksal bağ	paid = 1 AND refund = 0

NULL tuzağı
WHERE column = NULL yanlış mantıktır. NULL bilinmeyen değeri temsil eder; IS NULL / IS NOT NULL kullanılır.

5. ORDER BY ve LIMIT

```
SELECT customer_id, total_amount
FROM orders
WHERE status = 'completed'
ORDER BY total_amount DESC, order_date DESC
LIMIT 10;
```

ORDER BY sorgunun son çıktı sırasını tanımlar. LIMIT/TOP/FETCH sözdizimi veritabanına göre değişebilir. Analizde ilk 10 kayıt almak için LIMIT kullanmak mümkündür; model verisi üretirken ise sıra garantisi gerektiğinde mutlaka ORDER BY eklenmelidir.

6. Aggregate fonksiyonlar ve GROUP BY

Aggregate fonksiyonlar birden çok satırı özet değere dönüştürür. COUNT, SUM, AVG, MIN ve MAX en sık kullanılanlardır.

```
SELECT
  country,
  COUNT(DISTINCT customer_id) AS customers,
  COUNT(*) AS orders,
  SUM(total_amount) AS revenue,
  AVG(total_amount) AS aov
FROM orders
WHERE status = 'completed'
GROUP BY country;
```

Fonksiyon	Ne ölçer?	Dikkat
COUNT(*)	Satır sayısı	NULL fark etmez
COUNT(col)	NULL olmayan değer	NULL satırları sayılmaz
COUNT(DISTINCT col)	Benzersiz değer	Maliyetli olabilir
SUM	Toplam	Grain hatasına duyarlı
AVG	Ortalama	Ağırlıklandırma gerekebilir

7. HAVING - grupları filtreleme

```
SELECT customer_id,
    COUNT(*) AS order_count,
    SUM(total_amount) AS revenue
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >= 5
  AND SUM(total_amount) > 5000;
```

WHERE vs HAVING
WHERE gruplamadan önce satırları, HAVING ise GROUP BY sonrasında oluşan grupları filtreler.

8. CASE WHEN - SQL içinde iş kuralı

```
SELECT order_id, total_amount,
    CASE
      WHEN total_amount >= 5000 THEN 'VIP'
      WHEN total_amount >= 1000 THEN 'High'
      WHEN total_amount >= 250  THEN 'Medium'
      ELSE 'Low'
    END AS order_segment
FROM orders;
```

CASE; segmentasyon, flag üretme, conditional aggregation ve KPI tanımı için kritik bir araçtır.

```
SELECT
 SUM(CASE WHEN status = 'completed' THEN total_amount ELSE 0 END) AS completed_revenue,
 SUM(CASE WHEN refund = 1 THEN 1 ELSE 0 END) AS refunded_orders
FROM orders;
```

9. JOIN - tabloları güvenli biçimde birleştirme

JOIN	Ne döndürür?	Tipik kullanım
INNER JOIN	Her iki tabloda eşleşenler	Siparişi olan müşteriler
LEFT JOIN	Sol tablonun tümü + eşleşmeler	Siparişi olmayan müşterileri de tut
RIGHT JOIN	Sağ tablonun tümü	Nadiren gerekir
FULL OUTER JOIN	Her iki tarafın tümü	Mutabakat / veri kalite analizi
CROSS JOIN	Tüm kombinasyonlar	Takvim/deney tasarımı gibi özel durumlar

```
SELECT
  c.customer_id,
  c.country,
  o.order_id,
  o.total_amount
FROM customers c
LEFT JOIN orders o
 ON c.customer_id = o.customer_id;
```

JOIN sonrası doğrulama
Join öncesi ve sonrası benzersiz anahtar sayısını kontrol et. Örneğin customers tarafında customer_id tekilse, LEFT JOIN sonrasında aynı müşterinin çok satıra çıkması beklenen bir davranış mı, hata mı? Grain ile yanıtla.

10. JOIN cardinality: 1:1, 1:N ve N:N

İlişki	Örnek	Risk
1:1	customer ↔️ customer_profile	Düşük
1:N	customer ↔️ orders	Sol taraf çoğalır
N:1	orders ↔️ customer	Sipariş grain'i korunabilir
N:N	orders ↔️ promotions	Satırlar patlayabilir; bridge/pre-aggregation gerekebilir

```
-- Güvenli yaklaşım: önce item seviyesini order seviyesine özetle
WITH item_summary AS (
 SELECT order_id, SUM(quantity * unit_price) AS item_revenue
 FROM order_items
 GROUP BY order_id
)
SELECT o.order_id, s.item_revenue
FROM orders o
LEFT JOIN item_summary s USING (order_id);
```

11. Subquery

Subquery başka bir sorgunun içinde çalışan sorgudur. Filtreleme, karşılaştırma ve ara sonuç üretimi için kullanılır.

```
SELECT customer_id, total_amount
FROM orders
WHERE total_amount > (
  SELECT AVG(total_amount)
  FROM orders
);
```

- Scalar subquery: tek değer döndürür.
- IN/EXISTS subquery: üyelik veya varlık kontrol eder.
- Correlated subquery: dış sorgudaki satıra bağlı çalışır; pahalı olabilir.

12. CTE - karmaşık analizi parçalara ayırma

```
WITH customer_revenue AS (
  SELECT customer_id,
      SUM(total_amount) AS revenue
  FROM orders
  WHERE status = 'completed'
  GROUP BY customer_id
),
segments AS (
  SELECT customer_id, revenue,
      CASE WHEN revenue >= 5000 THEN 'VIP' ELSE 'Standard' END AS segment
  FROM customer_revenue
)
SELECT segment, COUNT(*) AS customers, AVG(revenue) AS avg_revenue
FROM segments
GROUP BY segment;
```

CTE ne zaman?
Bir analizde “önce bunu hesapla, sonra bunu kullan” cümlesi kuruyorsan CTE genellikle iyi bir seçimdir. Okunabilirliği artırır ve debug etmeyi kolaylaştırır.

13. EXISTS ve NOT EXISTS

```
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.customer_id
);
```

Bu sorgu hiç sipariş vermemiş müşterileri bulur. Varlık kontrolünde EXISTS çoğu zaman JOIN + DISTINCT yaklaşımından daha açık bir niyet taşır.

14. Window Functions - analitik SQL'in merkezi

Window function, satırları GROUP BY gibi tek satıra sıkıştırmadan bir pencere üzerinde hesap yapar. Veri analizi ve mülakatlarda en önemli SQL konularından biridir.

```
SELECT
  customer_id,
  order_id,
  order_date,
  total_amount,
  SUM(total_amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_revenue
FROM orders;
```

Fonksiyon	Amaç	Örnek
ROW_NUMBER()	Sıra numarası	Her müşterinin son siparişi
RANK()	Eşitlerde boşluklu rank	Ürün gelir sırası
DENSE_RANK()	Eşitlerde boşluksuz rank	Kategori sırası
LAG()	Önceki satır	Önceki sipariş tarihi
LEAD()	Sonraki satır	Sonraki event
SUM() OVER	Kümülatif/partition toplam	Running revenue
AVG() OVER	Hareketli ortalama	7 günlük ortalama

15. ROW_NUMBER ile "her grubun en son kaydı"

```
WITH ranked AS (
 SELECT o.*,
      ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY order_date DESC, order_id DESC
      ) AS rn
 FROM orders o
)
SELECT *
FROM ranked
WHERE rn = 1;
```

Mülakat klasiği
"Her müşterinin son siparişini getir", "kategori başına en yüksek 3 ürün", "ikinci en yüksek maaş" gibi soruların çoğu window function düşüncesini test eder.

16. Tarih ve zaman fonksiyonları

Tarih fonksiyonları veritabanına göre farklı isimlere sahip olabilir; ama analitik mantık aynıdır: dönem kırılımı, tarih farkı, rolling pencere ve cohort üretimi.

```
SELECT
  DATE_TRUNC('month', order_date) AS month,
  COUNT(*) AS orders,
  SUM(total_amount) AS revenue
FROM orders
GROUP BY 1
ORDER BY 1;
```

- DATE_TRUNC / YEAR-MONTH: haftalık/aylık aggregation.
- DATEDIFF / tarih çıkarma: müşteri yaşı, siparişler arası süre.
- INTERVAL / DATEADD: son 30 gün gibi dinamik pencereler.
- Timezone: event verilerinde UTC ↔️ lokal saat dönüşümünü açıkça yönet.

17. String ve NULL fonksiyonları

```
SELECT
  LOWER(TRIM(email)) AS normalized_email,
  COALESCE(country, 'Unknown') AS country,
  NULLIF(discount, 0) AS nonzero_discount
FROM customers;
```

Araç	Amaç
LOWER/UPPER	Metin standardizasyonu
TRIM	Boşluk temizleme
CONCAT	Metin birleştirme
SUBSTRING	Metin parçası
COALESCE	İlk NULL olmayan değer
NULLIF	Belirli değeri NULL yapma

18. Set işlemleri: UNION, UNION ALL, INTERSECT, EXCEPT

```
SELECT customer_id FROM web_customers
UNION ALL
SELECT customer_id FROM store_customers;
```

UNION duplicate satırları kaldırır, UNION ALL kaldırmaz. Performans ve niyet açısından fark önemlidir. INTERSECT ortak kayıtları, EXCEPT/MINUS ise bir tarafta olup diğerinde olmayanları bulur.

19. SQL'in mantıksal çalışma sırası

Sıra	Aşama	Zihinsel soru
1	FROM / JOIN	Hangi satırlar evrene giriyor?
2	WHERE	Hangi ham satırlar kalıyor?
3	GROUP BY	Hangi grain'e özetliyorum?
4	HAVING	Hangi gruplar kalıyor?
5	SELECT	Hangi kolon/metrikleri döndürüyorum?
6	DISTINCT	Tekrarları kaldırıyor muyum?
7	ORDER BY	Nasıl sıralıyorum?
8	LIMIT	Kaç satır gösteriyorum?

Neden önemli?
SELECT alias'ının WHERE içinde çoğu veritabanında kullanılamamasının nedeni mantıksal olarak WHERE aşamasının SELECT'ten önce çalışmasıdır.

20. Veri kalitesi için SQL kontrolleri

```
-- 1) Duplicate primary key
SELECT order_id, COUNT(*)
FROM orders
GROUP BY order_id
HAVING COUNT(*) > 1;

-- 2) Null oranı
SELECT
  AVG(CASE WHEN customer_id IS NULL THEN 1.0 ELSE 0 END) AS null_rate
FROM orders;

-- 3) Yetim foreign key
SELECT COUNT(*)
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

- Uniqueness: anahtar gerçekten tekil mi?
- Completeness: kritik alanlarda NULL oranı ne?
- Validity: amount < 0 gibi imkansız değer var mı?
- Referential integrity: foreign key karşılığı var mı?
- Freshness: en son kayıt tarihi beklenen kadar güncel mi?

21. Analitik SQL kalıpları

21.1 Müşteri bazında RFM özeti

```
WITH rfm AS (
 SELECT
  customer_id,
  MAX(order_date) AS last_order_date,
  COUNT(DISTINCT order_id) AS frequency,
  SUM(total_amount) AS monetary
 FROM orders
 WHERE status = 'completed'
 GROUP BY customer_id
)
SELECT * FROM rfm;
```

21.2 Günlük conversion rate

```
SELECT
 event_date,
 COUNT(DISTINCT CASE WHEN event_name = 'purchase' THEN user_id END) * 1.0
  / NULLIF(COUNT(DISTINCT CASE WHEN event_name = 'session_start' THEN user_id END), 0)
    AS conversion_rate
FROM events
GROUP BY event_date;
```

21.3 Funnel

```
SELECT
 COUNT(DISTINCT CASE WHEN event_name='view_product' THEN user_id END) AS viewers,
 COUNT(DISTINCT CASE WHEN event_name='add_to_cart' THEN user_id END) AS adders,
 COUNT(DISTINCT CASE WHEN event_name='purchase' THEN user_id END) AS buyers
FROM events;
```

21.4 Retention için first activity

```
WITH first_seen AS (
 SELECT user_id, MIN(event_date) AS cohort_date
 FROM events
 GROUP BY user_id
)
SELECT f.cohort_date,
     e.event_date - f.cohort_date AS day_number,
     COUNT(DISTINCT e.user_id) AS active_users
FROM first_seen f
JOIN events e ON e.user_id = f.user_id
GROUP BY 1,2;
```

22. Query performance ve optimizasyon sezgisi

Data Scientist'in DBA olması gerekmez; fakat gereksiz maliyet üreten sorguları tanıyabilmelidir. Özellikle cloud data warehouse ortamlarında taranan veri doğrudan maliyete dönüşebilir.

- SELECT * yerine gereken kolonları seç.
- Filtreleri mümkün olduğunca erken uygula.
- JOIN'den önce gerektiğinde pre-aggregate et.
- Fonksiyon uygulanmış filtre kolonlarının index/partition kullanımını bozabileceğini bil.
- Büyük tabloda gereksiz DISTINCT kullanma.
- Execution plan / EXPLAIN okumayı temel seviyede öğren.
- Partition pruning ve clustering mantığını bil.

```
EXPLAIN
SELECT customer_id, SUM(total_amount)
FROM orders
WHERE order_date >= '2026-01-01'
GROUP BY customer_id;
```

23. SQL dialect farkları

Konu	PostgreSQL / Snowflake benzeri	SQL Server / BigQuery notu
Satır limiti	LIMIT 10	TOP 10 veya LIMIT
Tarih kırpma	DATE_TRUNC	DATETRUNC / DATE_TRUNC
Tarih farkı	date - date / DATEDIFF	DATEDIFF / DATE_DIFF
String birleştirme	|| / CONCAT	+ / CONCAT
QUALIFY	Bazı sistemlerde var	BigQuery/Snowflake güçlü destek

Pratik kural
Mülakat ve işte SQL mantığı taşınabilir; fonksiyon isimleri değişebilir. Bir dialekti iyi bil, diğerlerine dokümantasyonla adapte ol.

24. Uçtan uca vaka: E-Commerce gelir neden düştü?

Senaryo: Yönetim, son ay gelirin önceki aya göre %12 düştüğünü söylüyor. Görevin düşüşün kaynağını SQL ile ayrıştırmak.

1. Toplam geliri ve sipariş sayısını aylık karşılaştır.
2. Geliri traffic source, country, device ve customer_type segmentlerine kır.
3. Revenue = orders × AOV mantığıyla düşüşün hacim mi sepet değeri mi olduğunu ayır.
4. Conversion funnel verisi varsa session → product view → cart → checkout → purchase aşamalarını incele.
5. İptal/refund artışı ve ödeme hata oranını kontrol et.
6. En büyük negatif katkıyı yapan segmentleri sıralayıp iş hipotezi üret.

```
WITH monthly AS (
 SELECT
  DATE_TRUNC('month', order_date) AS month,
  device,
  COUNT(DISTINCT order_id) AS orders,
  SUM(total_amount) AS revenue,
  AVG(total_amount) AS aov
 FROM orders
 WHERE status='completed'
 GROUP BY 1,2
),
lagged AS (
 SELECT *,
       LAG(revenue) OVER (PARTITION BY device ORDER BY month) AS prev_revenue
 FROM monthly
)
SELECT *,
     revenue - prev_revenue AS revenue_change
FROM lagged
ORDER BY month, revenue_change;
```

Beklenen çıktı

“Gelir düştü” demek analiz değildir. İyi analiz: “Mobil trafikte completed order sayısı %18 düştü; AOV yatay. Düşüşün %70’i mobil yeni müşterilerden geliyor ve payment_error oranı aynı dönemde 2.1% → 5.8% yükseldi.” gibi neden zinciri kurar.

25. Mülakatlarda sık çıkan SQL soru kalıpları

Soru tipi	Ana araç
Her müşterinin son siparişi	ROW_NUMBER / MAX + JOIN
İkinci en yüksek değer	DENSE_RANK / subquery
Ardışık gün aktif kullanıcı	LAG / tarih farkı
Kategori başına top 3 ürün	RANK / DENSE_RANK
7 günlük hareketli ortalama	AVG OVER + frame
Sipariş vermeyen müşteri	NOT EXISTS / LEFT JOIN IS NULL
Aylık büyüme	LAG + yüzde değişim
Duplicate bulma	GROUP BY + HAVING COUNT > 1
Cohort retention	MIN date + cohort grouping
Funnel conversion	Conditional DISTINCT counts

26. Mülakat çözüm çerçevesi

7. Soruyu tekrar tanımla ve çıktı grain'ini söyle.
8. Gerekli tabloları ve join anahtarlarını belirle.
9. Edge case'leri açıkla: NULL, duplicate, tie, timezone, cancelled orders.
10. Önce doğru ve okunabilir çözümü yaz; gerekirse optimize et.
11. Sonuç validasyonu için count/check sorguları söyle.

Güçlü aday davranışı
Sadece sorguyu yazmak değil, varsayımları açık etmek ve veri kalitesi risklerini söylemek seni öne çıkarır.

27. Alıştırmalar

12. Her şehir için toplam gelir, sipariş sayısı ve AOV hesapla.
13. Son 90 günde hiç sipariş vermeyen müşterileri bul.
14. Her müşterinin ilk ve son sipariş tarihini getir.
15. Her kategori için gelir açısından ilk 3 ürünü bul.
16. Aylık gelir büyüme oranını hesapla.
17. Bir müşterinin siparişleri arasındaki gün farkını LAG ile hesapla.
18. Duplicate order_id kayıtlarını tespit et.
19. Her ay yeni müşteri ve returning müşteri sayısını çıkar.
20. Checkout → purchase conversion rate'ini device bazında hesapla.
21. İade oranı en yüksek 5 ürün kategorisini bul.
22. 7 günlük rolling revenue üret.
23. Müşteri başına lifetime revenue ve order count çıkar.
24. Order_items ile orders birleşiminde revenue şişmesi olup olmadığını test et.
25. İlk siparişinden sonraki 30 gün içinde ikinci sipariş veren müşteri oranını hesapla.
26. A/B testi tablosunda grup bazında exposed users, converters ve conversion rate hesapla.

28. Bilmen gereken tek büyük SQL kalıbı

```
WITH base AS (
  SELECT ...
  FROM table_a a
  LEFT JOIN table_b b ON ...
  WHERE ...
),
features AS (
  SELECT
    ...,
    CASE WHEN ... THEN ... END AS ...,
    LAG(...) OVER (PARTITION BY ... ORDER BY ...) AS ...
  FROM base
),
summary AS (
  SELECT ...,
      COUNT(DISTINCT ...) AS ...,
      SUM(...) AS ...,
      AVG(...) AS ...
  FROM features
  GROUP BY ...
  HAVING ...
)
SELECT *
FROM summary
ORDER BY ... DESC;
```

Bu kalıp "her sorunun cevabı" değildir; fakat modern analitik SQL'de düşünce sırasını iyi özetler: doğru evren → temiz/feature edilmiş satırlar → aggregation → final çıktı.

29. SQL Cheat Sheet

Amaç	Kullan
Kolon seç	SELECT
Satır filtrele	WHERE
Sırala	ORDER BY
Özetle	GROUP BY + aggregate
Grupları filtrele	HAVING
İş kuralı	CASE WHEN
Tablo birleştir	JOIN
Ara sorgu	CTE / subquery
Varlık kontrolü	EXISTS / NOT EXISTS
Grup içinde sıra	ROW_NUMBER / RANK
Önceki/sonraki değer	LAG / LEAD
Kümülatif metrik	SUM OVER
Eksik değer	COALESCE / IS NULL
Tarih analizi	DATE_TRUNC / DATEDIFF
Set birleştir	UNION ALL

30. Final kontrol listesi

- ☐ Çıktının grain'ini tanımlayabiliyorum.
- ☐ Primary/foreign key ve join cardinality kontrolü yapabiliyorum.
- ☐ SELECT, WHERE, ORDER BY, GROUP BY, HAVING ve CASE kullanabiliyorum.
- ☐ INNER ve LEFT JOIN farkını sonuç satırları üzerinden açıklayabiliyorum.
- ☐ CTE ve subquery ile karmaşık analizi bölebiliyorum.
- ☐ ROW_NUMBER, RANK, LAG, LEAD ve aggregate window functions kullanabiliyorum.
- ☐ Tarih bazlı KPI ve rolling analiz yazabiliyorum.
- ☐ NULL, duplicate ve many-to-many join risklerini fark edebiliyorum.
- ☐ SQL ile funnel, retention, growth ve customer-level feature üretebiliyorum.
- ☐ Sorgu sonucunu kontrol eden validation query'leri yazabiliyorum.
- ☐ Mülakatta varsayımlarımı ve edge case'leri açıklayabiliyorum.

Modül tamamlanma kriteri
Bu listedeki maddeleri kopya çekmeden yapabiliyor ve 30-45 dakikada orta seviye bir analitik SQL vakasını çözebiliyorsan, veri bilimci için sağlam SQL temeline sahipsin.